

Embedded Software

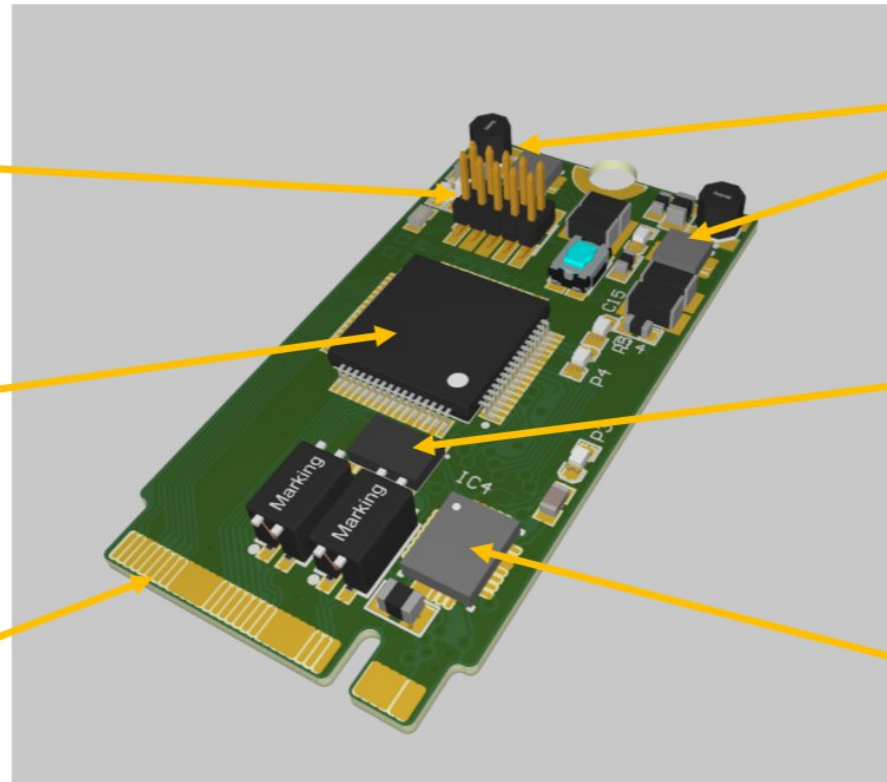
ED

Hardware (Processorboard)

**Programming
Connector**
- SWD

**stm32f405
RTGE**
- 192 kB RAM
- 1 MB Flash

**m.2
Slot**

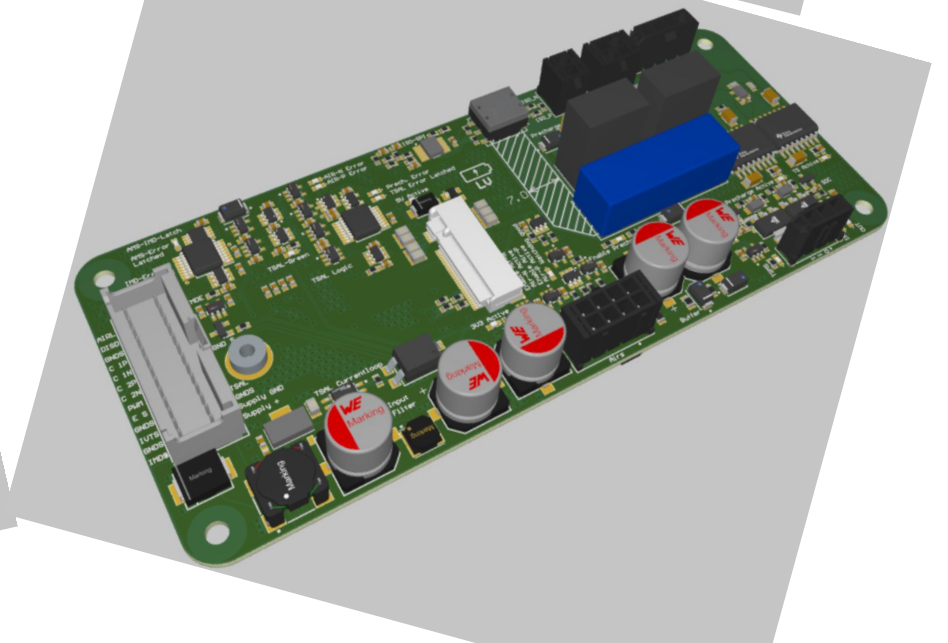
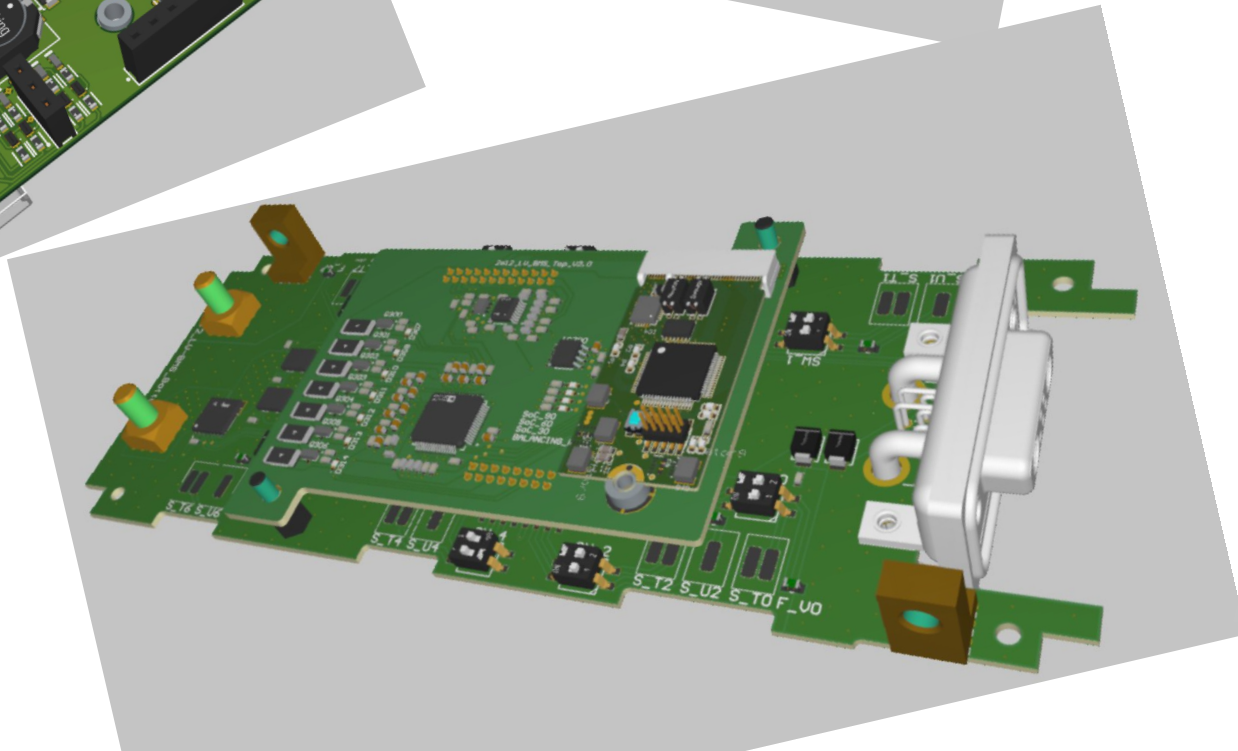
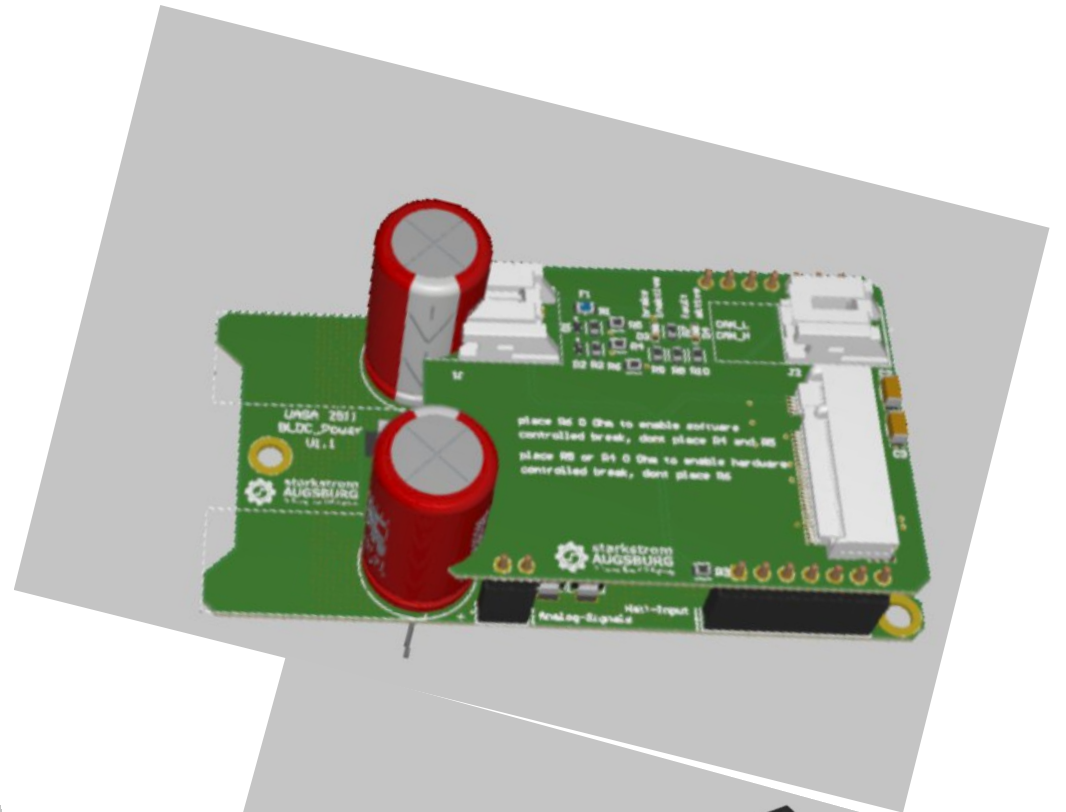
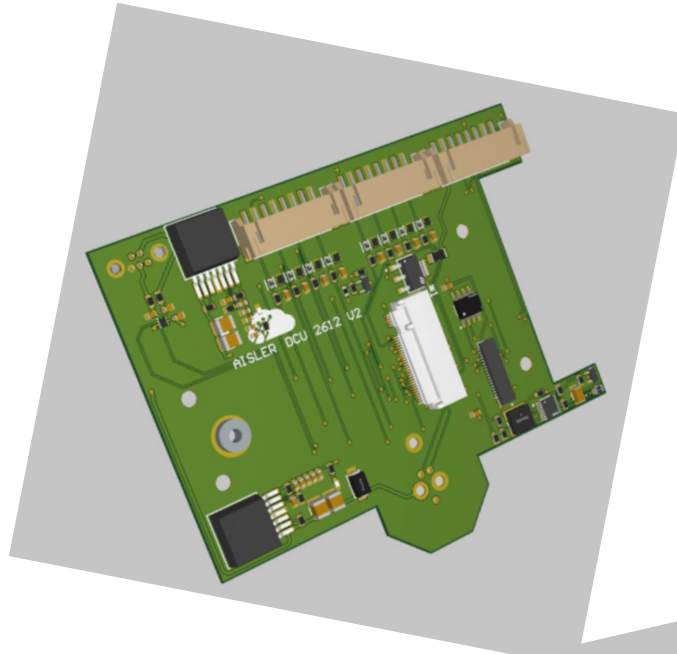
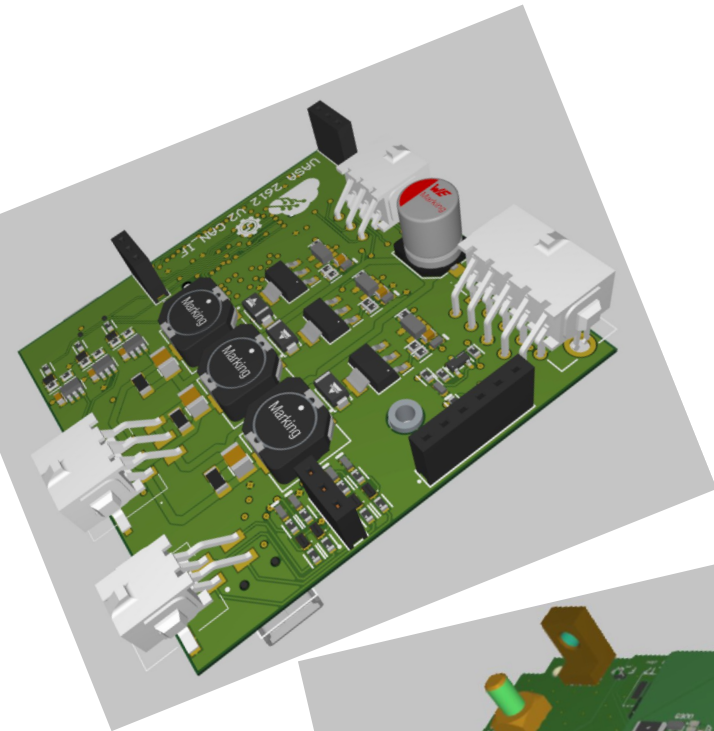


**Power
Supply**
- 5V
- 3V3

**2 x
CAN**

**E-
fuse**

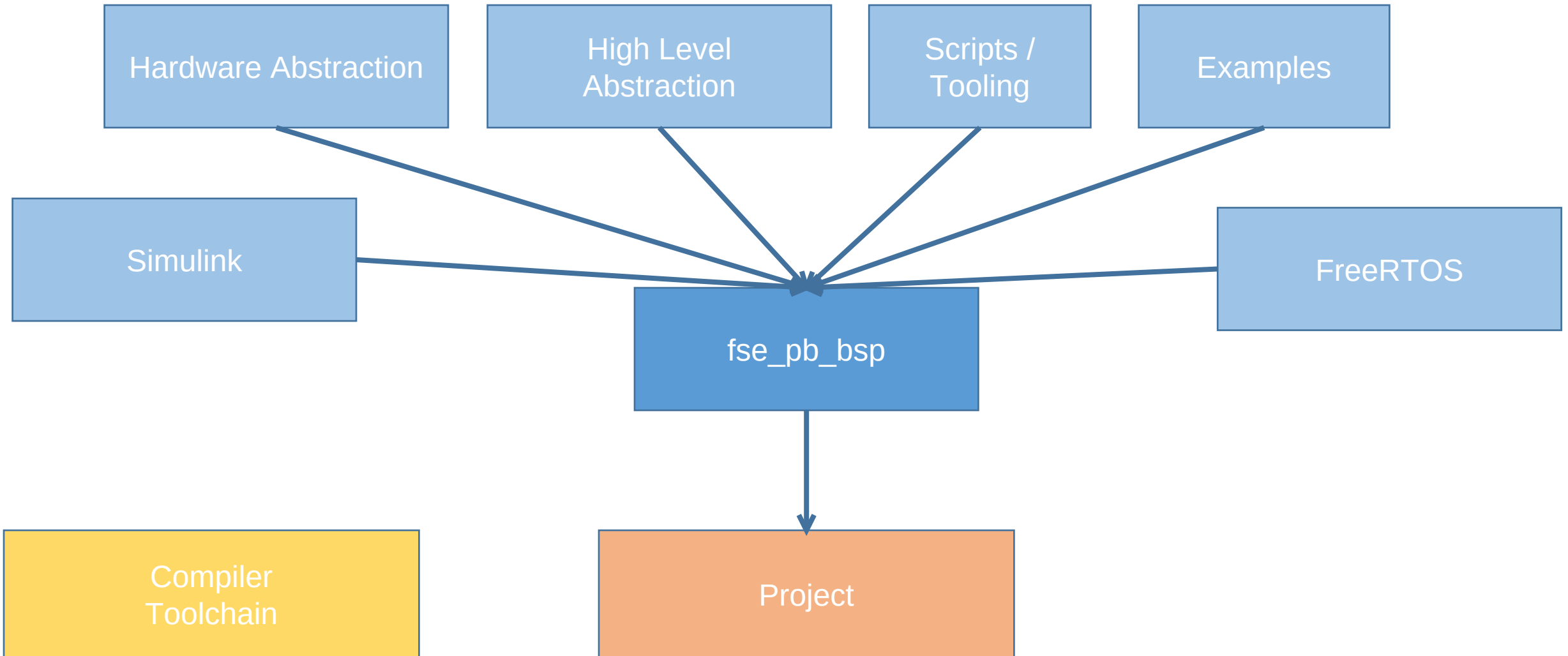
Usage



Usage

- HV Accumulator (BMAS)
- LV Accumulator
- Steering Controller
- DCU
- CAN Interface

Processorboard Board Support Package



Tooling

- Custom scripts for
 - Compiling
 - Flashing (over Debugger or CAN-Bus)
 - Debugging via OpenOCD
 - Viewing Example usage
 - Matlab (Simulink)

Tooling

```
(.venv) maximilian@fedora:~/Git/fse_can_interface$ ./ss
/home/maximilian/Git/fse_can_interface
usage: ss.py [-h]
            {flash,bootloader,canflash,flash_example,build,clean,example_build,example_list,oocd_start
...

```

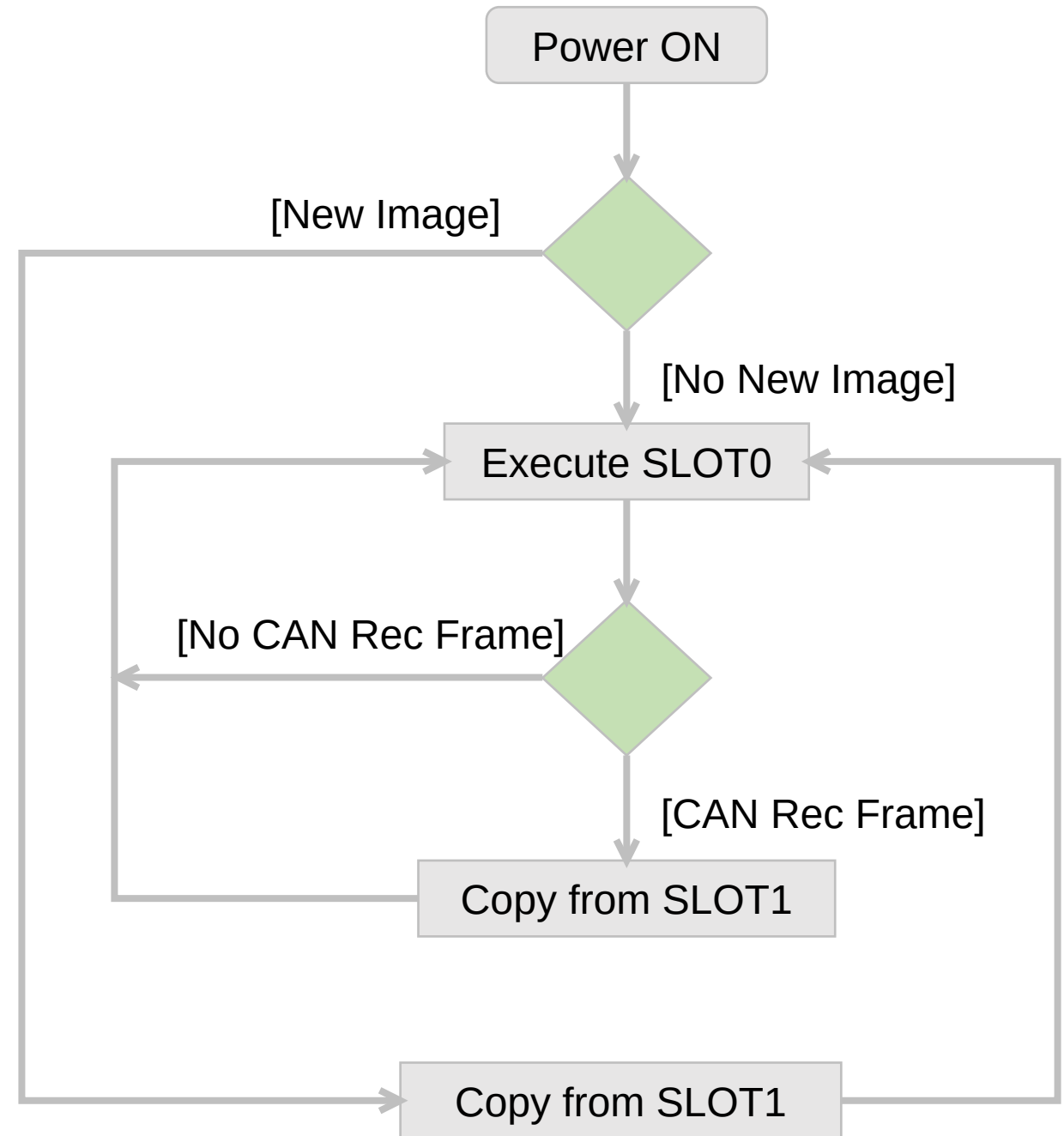
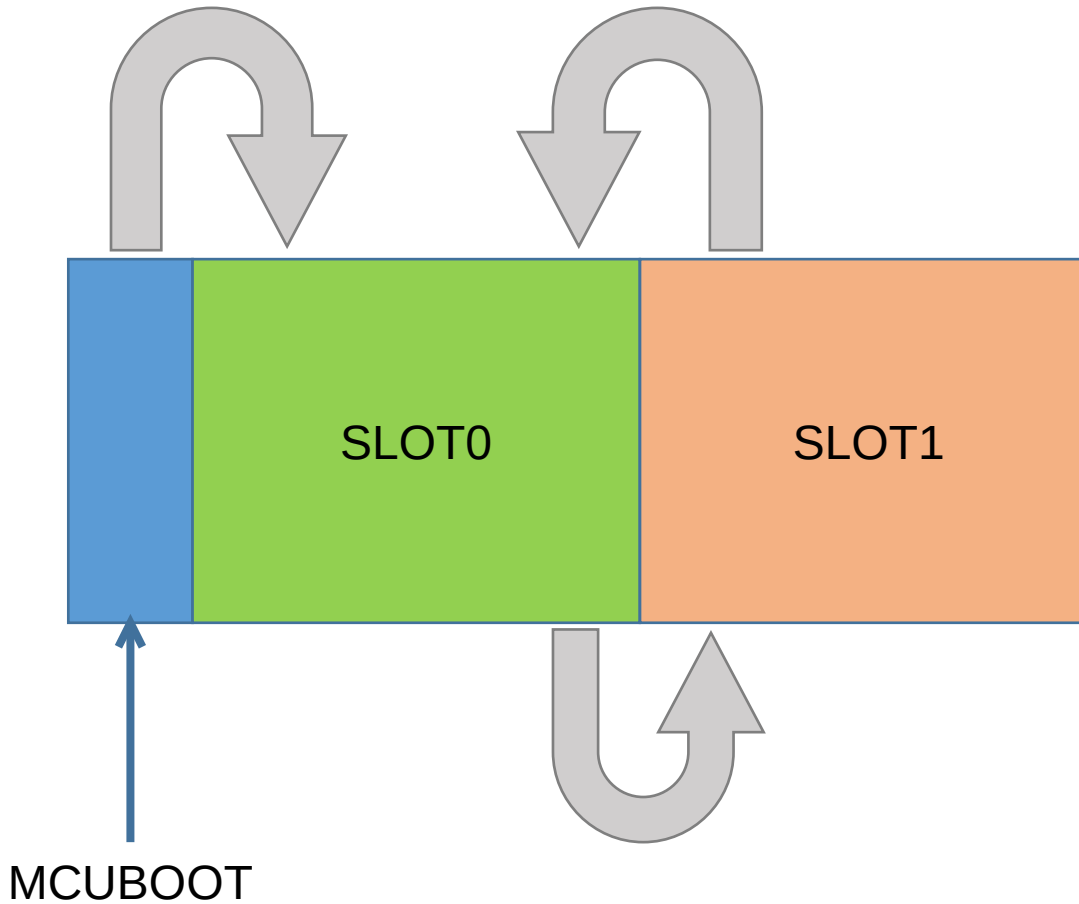
positional arguments:

<code>{flash,bootloader,canflash,flash_example,build,clean,example_build,example_list,oocd_start,oocd_stop,</code>	
<code>flash</code>	flash via telnet and stlink
<code>bootloader</code>	flash bootloader via telnet and stlink
<code>canflash</code>	flash via can
<code>flash_example</code>	build and flash one of the fse_pb_bsp examples via telnet and stlink
<code>build</code>	build program
<code>clean</code>	clean workspace
<code>example_build</code>	build one of the fse_pb_bsp examples (adc, pwm, can_tx, ...)
<code>example_list</code>	list available fse_pb_bsp examples
<code>oocd_start</code>	start oocd
<code>oocd_stop</code>	stop oocd
<code>oocd_state</code>	stop oocd
<code>matlab_init</code>	install matlab paths, build the ss block libraries and create the model
<code>matlab_build</code>	generate c code from the simulink model
<code>matlab_libs</code>	rebuild the ss block libraries only
<code>matlab_open</code>	open the simulink model in the matlab gui
<code>matlab_pins</code>	list the pin names the ss blocks accept
<code>matlab_cache_clean</code>	drop the library browser icon cache (run with matlab closed)
<code>matlab_clean</code>	remove generated code, caches and generated can libraries
<code>can_gen</code>	generate simulink message blocks from a dbc file

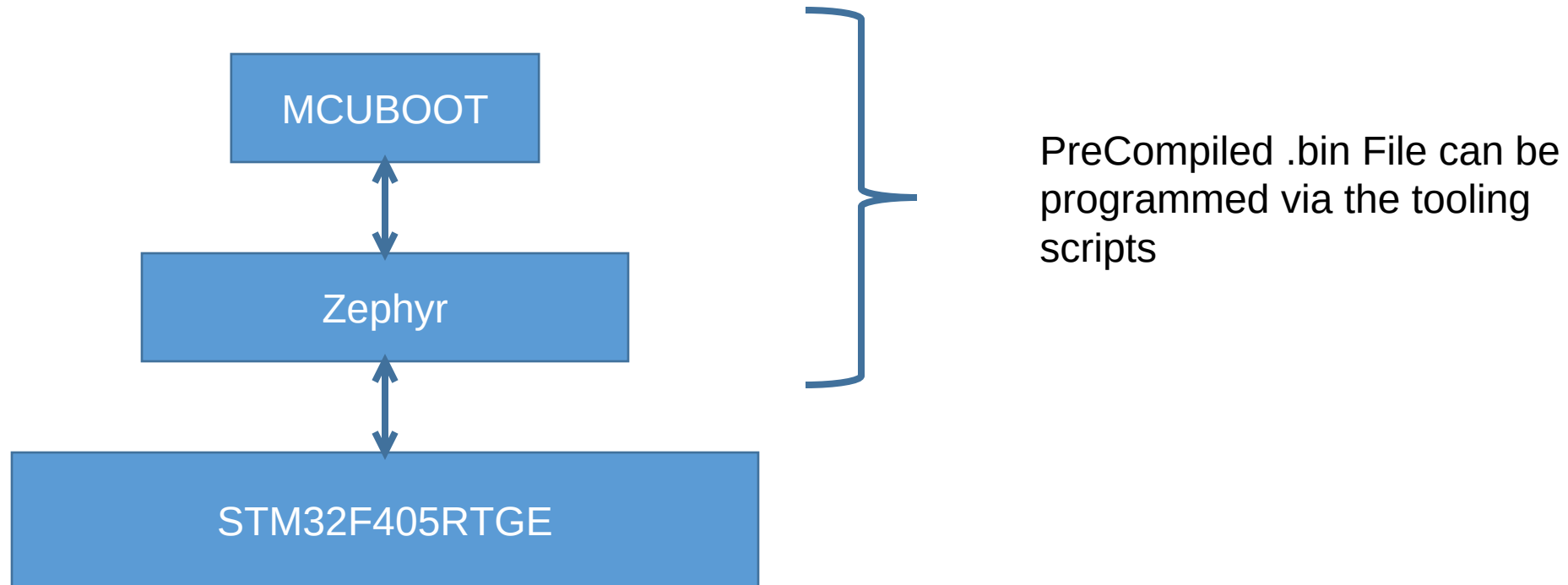
options:

<code>-h, --help</code>	show this help message and exit
-------------------------	---------------------------------

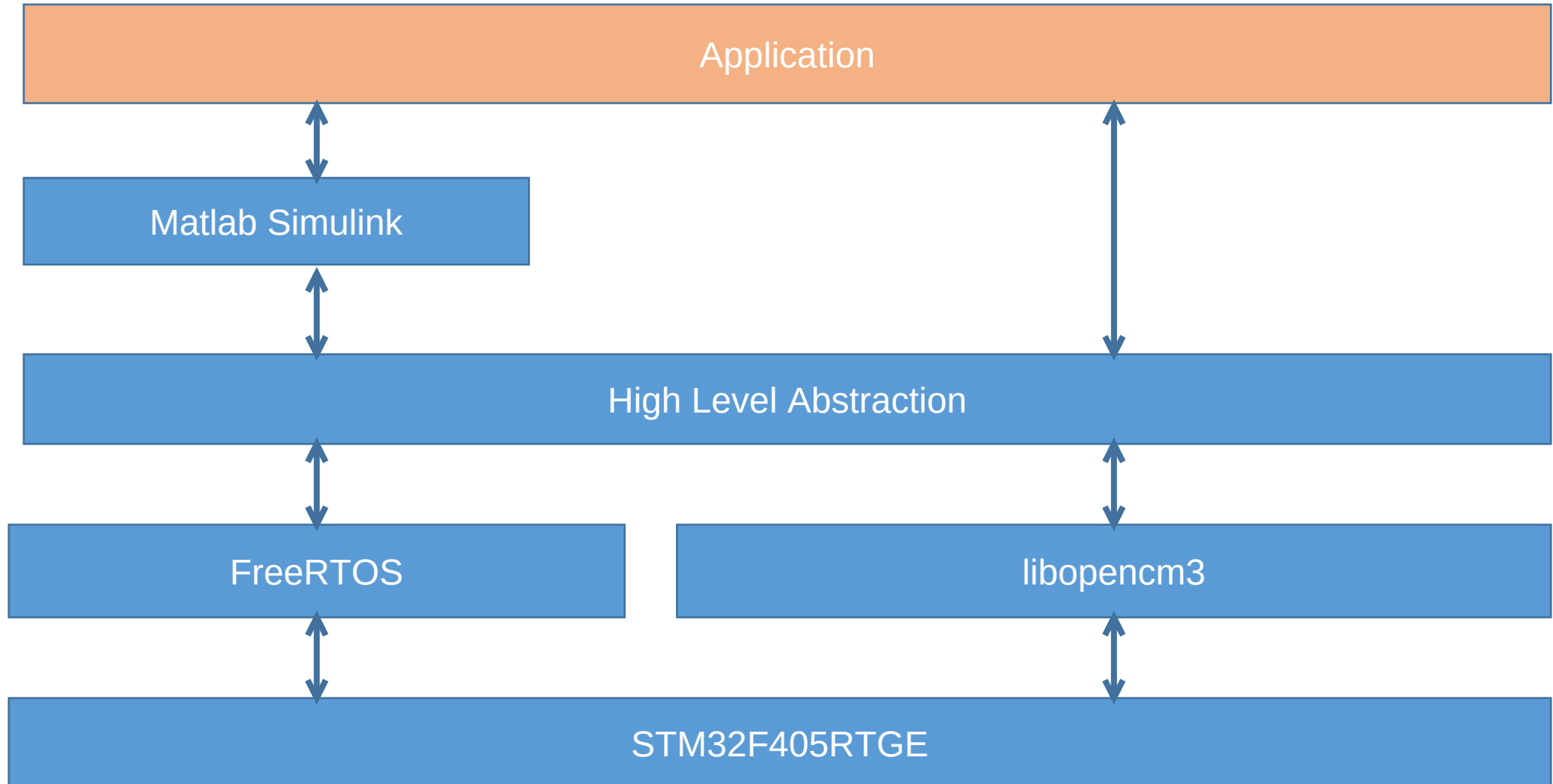
Bootloader



Software Stack (Bootloader)



Software Stack



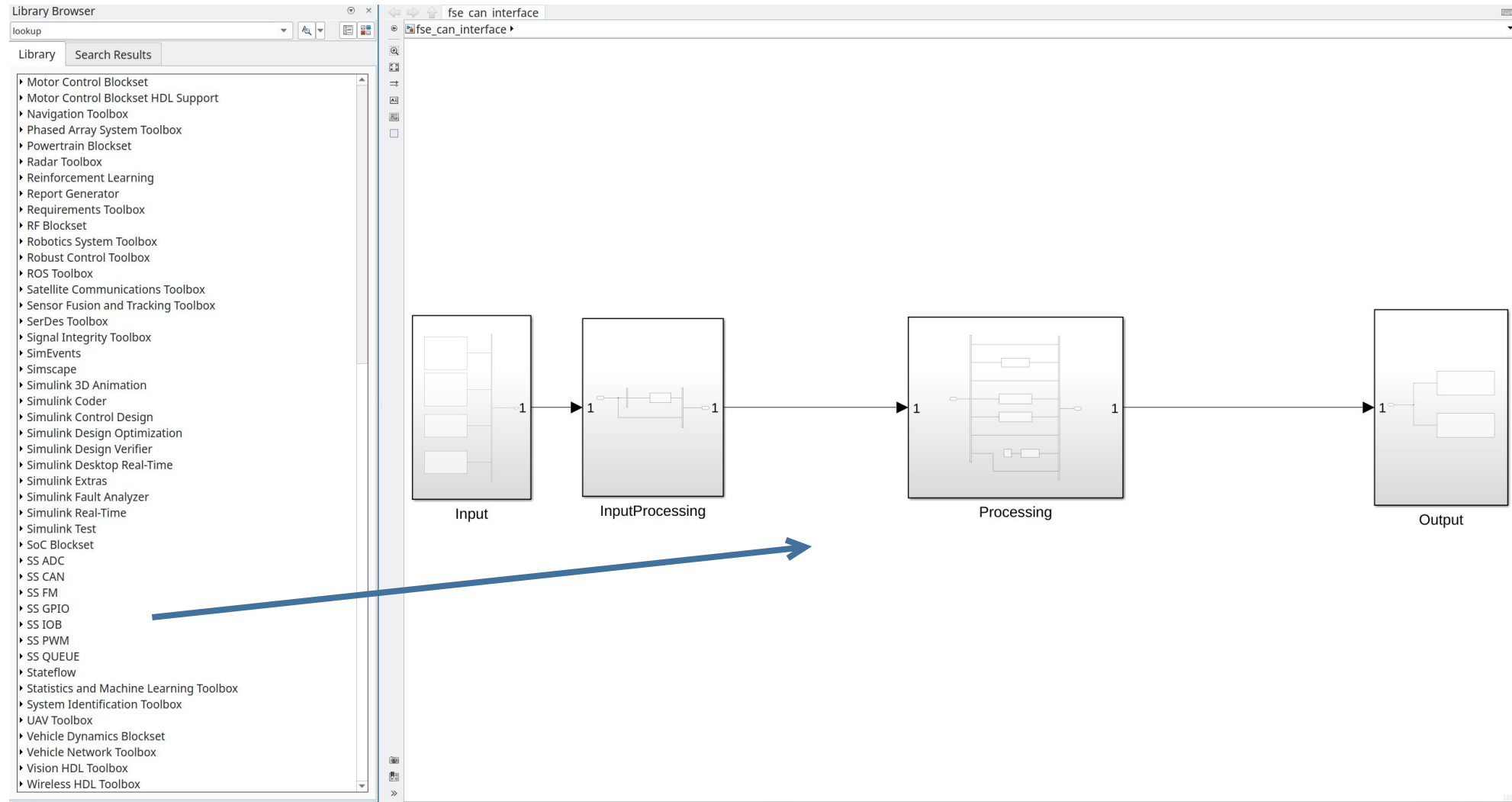
High Level Abstraction

- GPIO
- Timer based PWM
- SPI
- I2C
- ADC
- Interrupt based Input Observation
- Timer based frequency measurement
- CAN (With timer based timeout detection and CAN-Queues)
- Firmware update via CAN
- PID-Controller
- Filter abstraction (FIR and IIR)

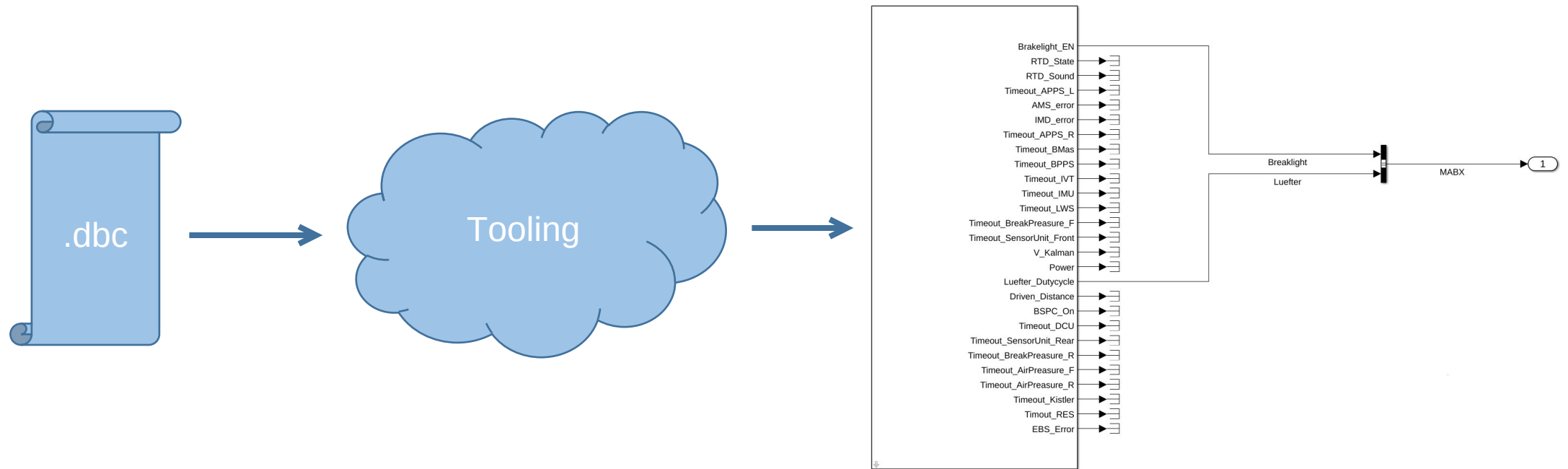
Matlab

- Integration of Simulink-SFunctions for
 - Input
 - Output
 - PWM
 - Frequency Measurement
 - Input Observation
 - ADC
 - CAN (with generated CAN-Blocks from .dbc-Files)

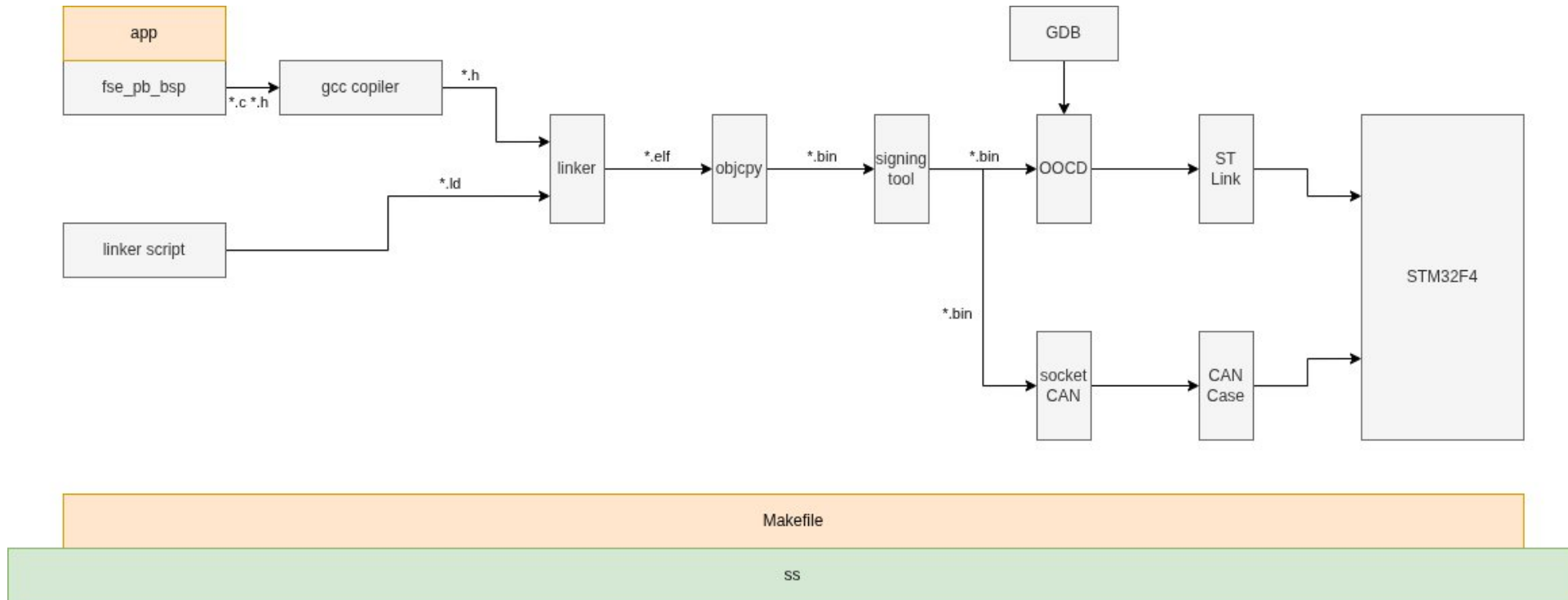
Matlab (Overview)



Matlab (CAN-Blocks)



Compile Flow (Linux)



Compile Flow

- The „Compile Flow“ is primarily developed for Linux
- The Tooling is extended to work under Windows too
 - STM-Cube-CLT is used
 - CAN-Flash is not supported because under Linux SocketCAN is used